

Variables: entier, flottant, booléen, chaîne de caractères, octet, liste, tuple

```
a=2 # a est un entier (integer:int)
b=3.5 # b est un flottant (float)
c="Python3" # c est une chaîne de caractères (string: str)
d=False # d est un booléen (bool)
e=b"FFR" # e est une variable de type octet (bytes)
f=[1,2,3,"dd"] # f est une liste de valeurs modifiables (list)
g=(3,5,9) # g est une liste de valeurs non modifiables (tuple)
```

Affectation de variables

```
x=3.2E+3 # notation scientifique
y=4+2*x # priorité au produit sur l'addition
a,b=3,5.0 # double affectation soit a = 3 b = 5.0
x+=3 # équivaut à x=x+3
z=0xA # notation hexadécimale 0x
y=0b0110 # notation binaire 0b
```

Opérations mathématiques sur entiers et flottants

```
x=11
y=x/2 # division y = 5.5
y=x//2 # division entière renvoie y=5
r=x%3 # modulo ou reste de la division entière renvoie r=2
z=x**3 # élévation à la puissance 3
n=abs(-3.5) # valeur absolue soit n = 3.5
m=pow(4,3) # m= 64 soit 4 à la puissance 3
```

Opérations logiques sur booléens

```
a=False
b=True
s= (a and b) or (a and not(b)) # s = 0.1 + 0.0 = 0
```

Conversions de types de variables

```
a=int("15") # renvoie un entier a=15
b=int(23.3) # renvoie un entier b=23
c=float(2) # renvoie un float c=2.0
d=str(22) #renvoie un str d="22"
e=round(23.3) # renvoie l'arrondi e=23
f=round(23.32,1) # renvoie l'arrondi avec une décimale f=23.3
g=chr(64) # renvoie g="@" selon le code ASCII
h=ord("@") # renvoie h=64 # selon le code ASCII
```

Opérations sur listes et tuples

```
k=[1,2.0,3,"dd"] # k est une liste hétérogène modifiable
a=k[0] # renvoie a=1 élément de la liste k d'index 0
fin=k[-1] # renvoie fin="dd" dernier élément de la liste k
L=len(k) # renvoie L=4 nombre d'éléments de la liste
p=k[1:3] # renvoie p=[2.0,3] liste des éléments indexés 1 à 3 exclu!
k.append(5) # ajout d'un élément 5 à la fin
k.remove(2.0) # suppression de l'élément de valeur 2.0
q=[2,5,3,2,4,5]
q.sort() # renvoie la liste q triée par ordre croissant q=[2,3,2,4,5,5]
q.reverse() # inverse l'ordre des éléments q=[4,5,3,2,5,2]
```

Saisie des entrées et affichage des sorties

```
a=input("Saisir un ou plusieurs caractères:") # a est un string
b=int(input("Saisir une valeur entière:")) # b est un entier
print("Votre chaîne de caractères est:",a) # la virgule est le séparateur
print("La valeur entière saisie est:",b)
print("prenom"+"."+nom) # concaténation renvoie prenom.nom
print("Terminé\n") # \n indique un saut de ligne
```

Opérations sur chaînes de caractères

```
s="Bla-bla"
L=len(s) #renvoie le nombre d'éléments de s L=7
print(s.upper()) # affiche en majuscules BLA-BLA sans les guillemets
print(s.lower()) # affiche en minuscules bla-bla
u=s.split("-") # renvoie une liste de deux éléments u=["Bla","bla"]
```

Définition et appel d'une fonction

```
def poly(a:float,b:float,c:float,x:float):# nom et types des variables
    # Calcule  $y = ax^2 + bx + c$  spécification de la fonction
    y=a*x*x + b*x +c # la variable y est locale
    return y # valeur calculée
z=poly(1,2,3,2) # appel de la fonction avec paramètres renvoie z=11
```

Boucle itérative inconditionnelle

```
for i in range(4): # pour i compris entre 0 et 3
    print(i**2) # affiche 0 1 4 9
for i in range(4,20,4): # pour i de 4 inclu à 20 exclu par pas de 4
    print(i) # affiche 4 8 12 16
```

Boucle itérative conditionnelle

```
masse=12
while masse<100: # : et indentation
    print(masse) # affiche 12 32 52 72 92 avec sauts de ligne
    masse=masse+20
```

Test conditionnel

```
a=3.2
if(int(a)<=4): # : et indentation
    print("Ok") # test vrai donc on affiche Ok
```

```
# test avec double condition
if(a>2 and a<5):
    print(a) # test vrai donc affiche 3.2
```

```
# Si ..... sinon.....
if(a>1):
    print("sup") # test vrai donc affiche sup
else:
    print("inf")
```

```
# Si ....Sinon si.....Sinon .....
if a>5:
    a=a+2
elif a<2:
    a=a+3
else: a=a+5 # test vrai donc a=8.2
print(a) # test vrai donc affiche 3.2
```

Importation de modules de bibliothèques

```
import numpy as np # importe tous les modules de la librairie numpy
y=np.sin(teta) # np est un alias équivalent à numpy
from numpy import sin,cos # importe les modules sin et cos de numpy
y=sin(teta)
```

Génération automatique de tableaux à une dimension ou vecteurs

```
from numpy import linspace
a=linspace(10,20,3) # renvoie un tableau à une dimension de 3 float compris
entre 10 inclus et 20 inclus soit a =[10.0,15.0,20.0]
from numpy import arange
z=arange(10,20,2) # renvoie un tableau à une dimension d'entiers espacés
de 2 compris entre 10 inclus et 20 exclus soit z =[10,12,14,16,18]
```

False, True, if, while, from sont des mots-clés reconnus par l'interpréteur
int(), print(), range() ... sont des fonctions natives de Python